Reinforcement Learning with Ensemble Learning

Simon Liu

Nov 5, 2026

Reinforcement Learning

Machine Learning Paradigms

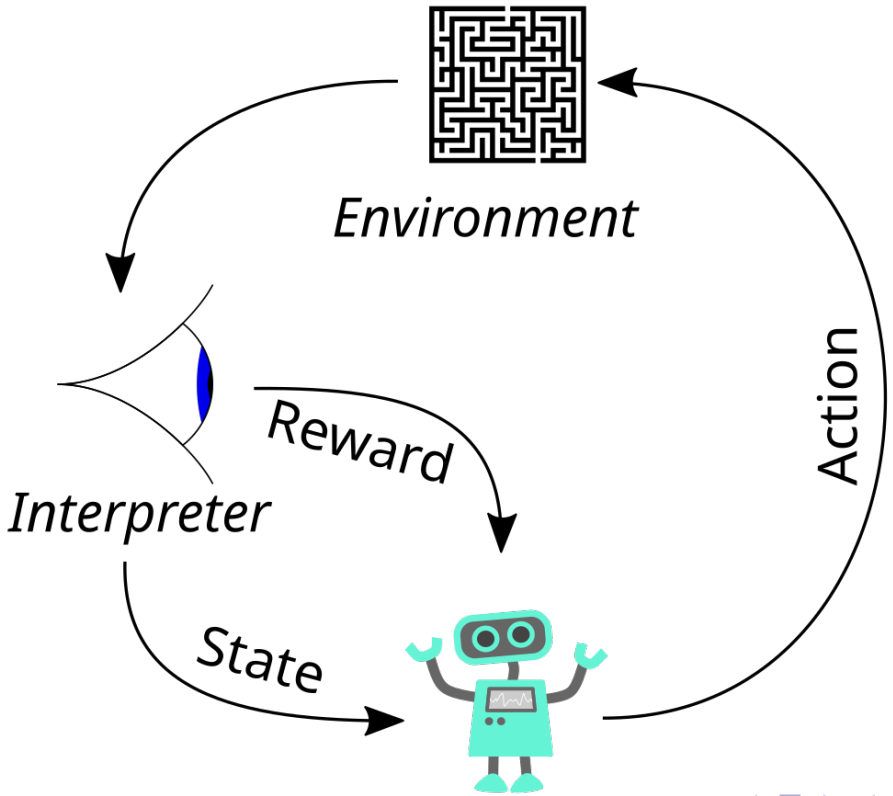
- **Supervised Learning:** Learn a mapping from inputs to outputs using labeled data (e.g., image classification, regression).
- **Unsupervised Learning:** Discover patterns or structure in unlabeled data (e.g., clustering, dimensionality reduction).
- **Reinforcement Learning:** Learn to make decisions by interacting with an environment and receiving feedback (rewards/penalties) to maximize cumulative reward.

Major Achievements in Reinforcement Learning

- **Atari Games:** Deep Q-Networks (DQN) reached human-level performance on various Atari 2600 games.
- **AlphaGo:** Combined RL and deep learning to defeat world champion Go players.
- **OpenAI Five:** Used RL to defeat professional Dota 2 teams in a complex, multi-agent environment.
- **AlphaStar:** Achieved grandmaster level in StarCraft II, demonstrating mastery in real-time strategy.
- **MuZero:** Learned optimal strategies in board games and Atari without prior knowledge of the rules.

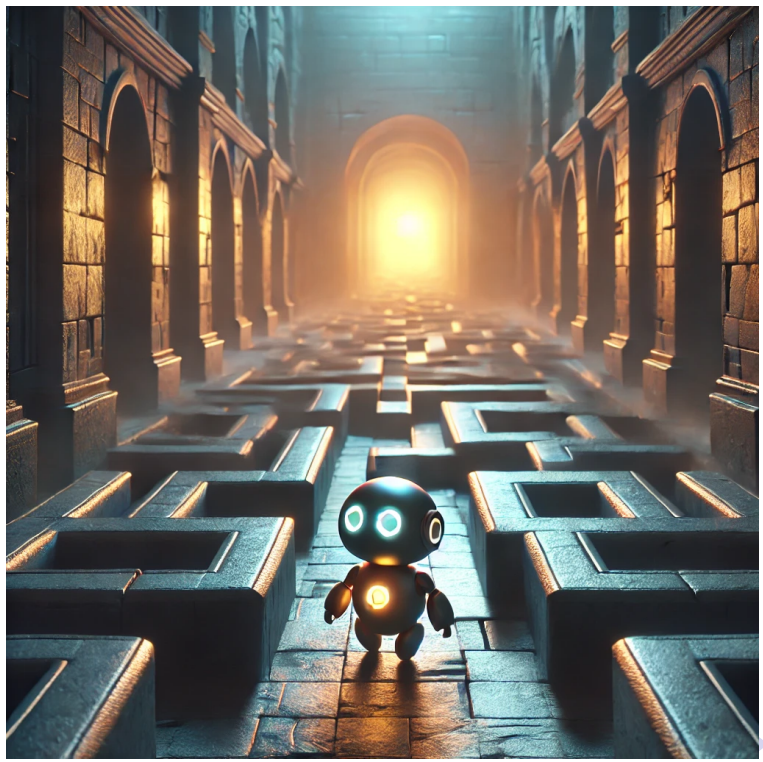
What is Reinforcement Learning? – Theory

- An RL system has an agent that learns to make decisions by interacting with an environment.
- Learning occurs via trial and error: the agent takes actions and receives rewards (or penalties).
- The objective is to maximize cumulative reward over time.



What is Reinforcement Learning? – Running Example

- Imagine a robot placed in an unfamiliar maze.
- The robot explores by moving in various directions.
- It receives a reward when it moves closer to the exit and a penalty when it collides with a wall.
- Over time, by learning from these outcomes, the robot refines its navigation strategy.



Key Components of Reinforcement Learning – Theory

- **Agent:** The decision maker that interacts with the environment.
- **Environment:** The external system the agent interacts with, providing observations and rewards.
- **State:** A representation of the current situation.
- **Action:** The set of possible moves the agent can take.
- **Reward:** The feedback signal received after an action, guiding learning.
- **Policy:** The strategy mapping states to actions, which the agent seeks to optimize.

Understanding these components is essential to model any RL problem.

Key Components of Reinforcement Learning – Running Example

- In our maze scenario:
  - **Agent:** The robot navigating the maze.
  - **Environment:** The maze itself, including corridors, walls, and the exit.
  - **State:** Each cell or position in the maze, representing where the robot currently is.
  - **Action:** The possible moves (e.g., up, down, left, right) available to the robot.
  - **Reward:**
    - Positive rewards for moves that bring the robot closer to the exit.
    - Negative rewards (penalties) for hitting walls or making inefficient moves.
  - **Policy:** The robot's strategy for choosing its moves, which it improves as it learns from the outcomes.

Here, the maze structure defines the states, while the robot learns the best actions to reach the exit.

Markov Decision Processes (MDPs) – Theory

- An MDP is a formal framework for modeling sequential decision-making under uncertainty.
- **Components of an MDP:**
  - **States (S):** All possible situations the agent can encounter.
  - **Actions (A):** All possible decisions or moves available.
  - **Transition Probabilities:** $P(s'|s, a)$, the likelihood of reaching state s' when taking action a in state s.
  - **Reward Function:** $R(s, a, s')$, which specifies the reward received after the transition.
  - **Discount Factor:** γ, a value between 0 and 1 that prioritizes immediate rewards over distant ones.
- The **Markov Property** asserts that the next state depends only on the current state and action.

This framework is central to formulating and solving RL problems.

Markov Decision Processes (MDPs) – Running Example

- In our maze:
  - **States:** Each cell in the maze represents a state where the robot can be.
  - **Actions:** The robot chooses from moving up, down, left, or right.
  - **Transitions:** The outcome of an action—moving to an adjacent cell or staying if blocked—is defined probabilistically.
  - **Reward Function:**
    - The robot receives a positive reward when a move brings it closer to the exit.
    - It receives a negative reward if it collides with a wall or takes an inefficient path.
  - **Discount Factor:** Future rewards are weighted less, emphasizing immediate progress.
- The maze satisfies the Markov Property: the next cell depends only on the robot's current cell and chosen move.

Thus, the maze environment is modeled as an MDP, making it ideal for applying RL techniques.

Formulating the RL Problem – Theory

- In reinforcement learning, the goal is to maximize cumulative reward over time.
- The cumulative reward (or return) is defined as:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2\, r_{t+3} + \cdots$$

- Here, r_{t+1} is the immediate reward and γ (a discount factor between 0 and 1) determines how future rewards are valued relative to immediate rewards.
- In plain terms, the agent strives to balance short-term costs with long-term benefits.

Formulating the RL Problem – Running Example

- Consider our robot in a maze:
  - The robot's goal is to reach the exit while maximizing its overall reward.
  - Each move may incur a small penalty (e.g., -1) to encourage finding the shortest path.
  - Reaching the exit gives a substantial positive reward (e.g., $+10$).
- For example, a sample path might yield:

$$G_t = -1 + \gamma(-1) + \gamma^2(10)$$

- This return tells the robot the overall benefit of following that particular path.

Value Functions and the Q Function – Theory

- **Value Functions** estimate how good it is to be in a state or to take a certain action.
- The state-value function, $V(s)$, is defined as:

$$V(s) = \mathbb{E}[G_t \mid s_t = s]$$

  meaning the expected cumulative reward starting from state s.
- The action-value function (or Q function), $Q(s, a)$, is defined as:

$$Q(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a]$$

  which is the expected cumulative reward for taking action a in state s.
- Simply put, $V(s)$ tells us how promising a state is, and $Q(s, a)$ tells us how beneficial a particular action is in that state.

Value Functions and the Q Function – Running Example

- In our maze:
  - $V(s)$ represents the "value" of a cell—cells near the exit tend to have higher values.
  - $Q(s, a)$ tells the robot the expected benefit of taking a specific move (e.g., moving right) from that cell.
- As the robot explores, it learns:
  - Which cells are more promising (higher $V(s)$).
  - Which moves from a cell yield the best long-term outcomes (higher $Q(s, a)$).
- This knowledge guides the robot toward the exit over time.

The Bellman Equations – Theory

- The Bellman equations provide a recursive way to express value functions.
- For the state-value function, the Bellman equation is:

$$V(s) = \mathbb{E}\left[r_{t+1} + \gamma\, V(s_{t+1}) \mid s_t = s\right]$$

  This states that the value of a state is the immediate reward plus the discounted value of the next state.
- For the optimal Q function:

$$Q(s,a) = \mathbb{E}\left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') \mid s_t = s, a_t = a\right]$$

- In essence, the Bellman equations tell us that the "goodness" of a state or action is built upon its immediate benefit and the best possible future benefits.

The Bellman Equations – Running Example

- In our maze, imagine the robot is in a specific cell:
  - The value $V(s)$ of that cell equals the immediate reward (say, -1 for a move) plus the discounted value of the cell it transitions into.
  - For instance, if moving right gives an immediate reward of -1 and the next cell is estimated to have a value of 5, then:

$$V(s) \approx -1 + \gamma \times 5$$

- Similarly, $Q(s, a)$ estimates the benefit of taking a specific action (like moving right) from that cell.
- This recursive relationship helps the robot iteratively learn the best path to reach the exit.

Dynamic Programming Methods – Theory

- Dynamic Programming (DP) methods assume full knowledge of the environment's dynamics.
- They compute the optimal value function by iteratively applying the Bellman equation until convergence.
- **Value Iteration Update:**

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s, a) \Big[R(s, a, s') + \gamma \, V_k(s') \Big]$$

- In plain language, DP "sweeps through" all states and actions to determine the best long-term reward for each state.

Dynamic Programming Methods – Running Example

- Consider our robot in a maze where the full layout is known.
- The robot updates the value of each cell by considering every possible move from that cell.
- For each cell, it calculates the expected reward for each move and selects the highest value.
- Over multiple iterations, the value of each cell converges to the true long-term reward, revealing the optimal path to the exit.

Monte Carlo Methods – Theory

- Monte Carlo methods estimate value functions by averaging the returns over many complete episodes.
- They do not require a model of the environment; instead, they learn directly from sampled experiences.
- The state-value function is approximated as:

$$V(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)}$$

where:

- $G_t^{(i)}$ is the cumulative reward observed in the ith episode starting from state s,
- $N(s)$ is the number of episodes that visit state s.

- This approach is straightforward but may require many episodes for accurate estimates.

Monte Carlo Methods – Running Example

- In our maze, the robot explores the maze from various starting positions.
- For each cell, it records the total reward (or "return") earned when it eventually reaches the exit.
- By averaging the returns from $N(s)$ visits to that cell, the robot estimates the cell's value $V(s)$.
- This average gives the robot a sense of which areas of the maze are more promising for reaching the exit.

Temporal Difference (TD) Learning – Theory

- TD learning updates value estimates based on one-step transitions, combining ideas from DP and Monte Carlo.
- The basic TD update rule is:

$$V(s) \leftarrow V(s) + \alpha\Big(r + \gamma\, V(s') - V(s)\Big)$$

where:

  - α is the learning rate,
  - r is the immediate reward,
  - s' is the next state (the state after taking an action from s),
  - $V(s')$ is the value of the next state.

- In simple terms, TD learning allows the agent to update its belief about the current state using the immediate reward and the estimated value of the next state.

Temporal Difference (TD) Learning – Running Example

- For our robot in the maze, TD learning means updating the value of the current cell immediately after taking a step.
- For example, if the robot moves from cell s to cell s' and receives a reward r, it updates $V(s)$ using:

$$V(s) \leftarrow V(s) + \alpha\Big(r + \gamma\, V(s') - V(s)\Big)$$

- Here, s' represents the cell the robot enters next.
- This incremental update helps the robot gradually refine its understanding of which cells lead to the exit.
- Over time, these TD updates enable the robot to learn an effective navigation strategy even without waiting for full episodes to complete.

Q-learning Algorithm – Theory

- Q-learning is a model-free RL algorithm.
- It learns an action-value function, $Q(s, a)$, which estimates the expected cumulative reward for taking action a in state s.
- The update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\left(r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right)$$

- Here:
  - α is the learning rate.
  - r is the immediate reward.
  - γ is the discount factor.
  - s' is the next state.
- In simple terms, Q-learning updates its estimates by considering the immediate reward plus the best future reward.

Q-learning Algorithm – Running Example

- For our robot in the maze:
  - When the robot takes a move (e.g., moving right) from its current cell s, it receives an immediate reward r (such as -1 for a step or a penalty for hitting a wall).
  - It then observes the next cell s' and evaluates the Q-values for all possible moves from s'.
  - The Q-value for the move it just took is updated as:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big)$$

- Over time, the robot learns which moves lead to higher long-term rewards, guiding it toward the exit.

Deep Reinforcement Learning: DQN – Theory

- Deep Q-Networks (DQN) use a deep neural network to approximate the Q-value function $Q(s, a)$.
- This approach allows RL to scale to large or continuous state spaces.
- Key techniques include:
  - **Experience Replay:** Past experiences are stored and sampled randomly for training, reducing correlations between consecutive samples.
  - **Target Networks:** A separate network with parameters θ^- computes target Q-values and is updated less frequently for stability.
- The loss function minimized in DQN is:

$$L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$$

- In layman's terms, DQN uses a neural network to predict long-term rewards for actions, and techniques like experience replay and target networks help stabilize this learning.

Deep Reinforcement Learning: DQN – Running Example

- In our maze:
  - The robot now uses a neural network that takes its current location (or a representation of the maze) as input and outputs Q-values for each possible move.
  - With experience replay, the robot stores its past moves and outcomes, then randomly re-trains on these samples.
  - A target network is used to compute stable target Q-values, ensuring that the robot's learning updates are gradual.
- This method helps the robot generalize its learning across similar maze areas, even in complex or large mazes.

Challenges in Reinforcement Learning – Theory

- RL faces several practical challenges:
  - **Exploration vs. Exploitation:** Balancing between trying new actions and using known rewarding actions.
  - **High Variance:** Reward estimates can fluctuate, leading to unstable learning.
  - **Overestimation Bias:** Algorithms may overestimate action values, resulting in suboptimal decisions.
  - **Sample Efficiency:** RL often requires many interactions with the environment to learn effectively.
  - **Computational Complexity:** Large state spaces and deep models can be demanding.
- These issues require careful algorithm design and sometimes motivate the use of stabilization or ensemble techniques.

Challenges in Reinforcement Learning – Running Example

- For our robot in the maze:
  - The robot must decide whether to explore unknown parts of the maze or exploit paths it already knows lead toward the exit.
  - Early on, its reward estimates for different moves can vary widely, making it hard to know the best move.
  - The robot might overestimate the value of a path if it gets lucky a few times.
  - It needs many maze runs (samples) to accurately estimate the value of each cell and move.
- These challenges illustrate why RL algorithms must carefully balance exploration and exploitation and handle variance in their learning process.

Exploration Strategies in RL – Theory

- In reinforcement learning, the agent must balance exploration (trying new actions) with exploitation (using known rewarding actions).
- A common strategy is ϵ-greedy:
  - With probability ϵ, choose a random action.
  - With probability $1 - \epsilon$, choose the best-known action.
- Mathematically, the probability of selecting a random action is ϵ.
- This strategy helps the agent discover new possibilities while still leveraging learned information.

Exploration Strategies in RL – Running Example

- In our maze, the robot must sometimes try new paths instead of always following the current best-known route.
- For example, if $\epsilon = 0.1$, then 10% of the time the robot will randomly choose a direction.
- This randomness allows the robot to potentially discover shortcuts or less obvious routes to the exit.
- Balancing exploration and exploitation is key to finding the optimal path in the maze.

Overestimation Bias and Variance in RL – Theory

- **Overestimation Bias:** When using the max operator in Q-learning, noisy Q-value estimates may be overly optimistic.
- Formally:

$$\mathbb{E}\left[\max_a Q(s,a)\right] \geq \max_a \mathbb{E}\left[Q(s,a)\right]$$

- **Variance:** The degree to which Q-value estimates fluctuate can lead to instability.
- High variance and overestimation can cause the agent to select suboptimal actions.

Overestimation Bias and Variance in RL – Running Example

- In our maze, the robot's estimates for different moves in a cell can be noisy.
- When it picks the move with the highest estimated Q-value, that value might be over-optimistic.
- This overestimation bias may cause the robot to repeatedly choose a suboptimal path.
- High variance in estimates can make the robot's decisions erratic, hindering consistent navigation.
- Techniques like Double Q-learning or ensemble methods can help reduce these issues.

Ensemble Learning in Reinforcement Learning

Introduction to Ensemble Strategies in RL – Theory

- Ensemble methods combine multiple models to improve prediction accuracy and stability.
- In reinforcement learning, ensembles help reduce variance and mitigate overestimation
- Today we'll explore several ensemble strategies applied to RL:
  - Q-function ensembles
  - Reward ensembles
  - Loss function ensembles
  - Combination of models
  - Combination of training algorithms
  - Decision strategies
  - Ensemble DQN

Q-function Ensembles – Theory

- Q-function ensembles merge several estimates of the Q-function $Q(s, a)$ to form a more robust prediction.
- By combining multiple estimates (e.g., via averaging or majority voting), the ensemble reduces noise and mitigates overestimation bias.
- The aggregated Q-value is given by:

$$Q_{\text{agg}}(s, a) = \frac{1}{K} \sum_{i=1}^{K} Q_i(s, a)$$

where K is the number of estimates.

Q-function Ensembles – Running Example

- In our maze, the robot maintains several separate estimates of the expected long-term reward for each possible move.
- Each estimate might be influenced by different experiences or training variations.
- The robot aggregates these estimates (for example, by averaging) to decide which move is most promising.
- This ensemble approach helps the robot avoid being misled by any single, noisy estimate, leading to more reliable navigation.

Reward Ensembles – Theory

- Reward ensembles combine different reward signals into a single, balanced feedback.
- For instance, one reward signal might measure progress toward the exit, while another penalizes collisions.
- The combined reward can be computed as a weighted sum:

$$R_{\text{combined}} = \lambda_1 R_1 + \lambda_2 R_2$$

- This approach smooths out noise and captures multiple aspects of performance.

Reward Ensembles – Running Example

- In our maze, the robot may receive a bonus for moving closer to the exit and a penalty for hitting walls.
- By combining these rewards, it gets a comprehensive signal that reflects both progress and caution.
- This balanced reward signal helps the robot decide which moves are truly beneficial.

Loss Function Ensembles – Theory

- Loss function ensembles combine multiple loss components to guide learning.
- One component might focus on immediate prediction errors, while another emphasizes long-term planning.
- The overall loss is a weighted sum:

$$L = \lambda_1 L_{\text{immediate}} + \lambda_2 L_{\text{long-term}}$$

- This combined loss helps stabilize training by balancing short-term and long-term objectives.

Loss Function Ensembles – Running Example

- For our robot, a combined loss means learning to both avoid immediate obstacles and plan efficient routes.
- The immediate loss helps the robot react properly (e.g., avoid walls).
- The long-term loss encourages it to choose paths that lead steadily toward the exit.
- Together, these loss components enable a balanced and effective navigation strategy.

Combination of Models – Theory

- Combining models aggregates outputs from multiple independent RL models.
- Methods include averaging, majority voting, or weighted sums.
- This diversity in predictions yields more robust and stable value estimates.

Combination of Models – Running Example

- In our maze, the robot runs several independent navigation models.
- Each model recommends a move based on its learned experiences.
- By averaging these recommendations, the robot forms a consensus decision.
- This results in more reliable navigation through the maze.

Combination of Training Algorithms – Theory

- Different training algorithms can learn varied strategies from the same environment.
- For example, one model might be trained using standard Q-learning, while another uses a variant like SARSA.
- Aggregating these outputs (via weighted sums) results in a more balanced, robust learning process.

Combination of Training Algorithms – Running Example

- In our maze, suppose one model learns quickly (e.g., via Q-learning) and another learns more cautiously (e.g., via SARSA).
- Each model captures different aspects of the maze's dynamics.
- By combining their outputs, the robot benefits from both rapid adaptation and careful evaluation.
- This leads to improved decision making in complex maze areas.

Decision Strategies – Theory

- Decision strategies determine how to select the final action from multiple model outputs.
- Common methods include:
  - Majority voting: Choosing the most frequently recommended action.
  - Averaging: Selecting the action with the highest average Q-value.
  - Weighted voting: Assigning different weights based on model confidence.
- These methods help balance exploration and exploitation in decision making.

Decision Strategies – Running Example

- In our maze, the robot receives several suggested moves from its ensemble.
- It may use majority voting—if most models recommend moving right, it moves right.
- Alternatively, averaging Q-values may yield the highest value for one move.
- This ensemble decision-making leads to more stable and reliable navigation.

Ensemble DQN – Theory

- Ensemble DQN uses multiple deep network estimates of the Q-function.
- Each network is trained on different subsets of experience or initialized differently.
- The aggregated Q-value is computed as:

$$Q_{\text{ensemble}}(s, a) = \frac{1}{K} \sum_{i=1}^{K} Q_i(s, a)$$

- This approach improves exploration and reduces overestimation bias.

Ensemble DQN – Running Example

- In our maze, the robot employs several deep networks to predict long-term rewards.
- Each network, trained on varied experiences, provides its estimate for each possible move.
- By aggregating these estimates (e.g., averaging), the robot chooses the best move.
- This ensemble strategy enhances the robot's reliability in navigating complex maze sections.

Summary of Reinforcement Learning

- Reinforcement Learning seeks to maximize cumulative reward by learning optimal policies.
- Key components include: agent, environment, state, action, reward, and policy.
- Fundamental methods such as Q-learning and DQN form the basis of RL.
- Challenges include exploration, high variance, and overestimation bias.
- Our robot-in-a-maze example illustrated these RL principles in practice.

Summary of Ensemble Methods in RL

- Ensemble strategies combine multiple estimates, rewards, losses, models, and training approaches.
- Benefits include reduced variance, mitigated overestimation bias, and more robust decision making.
- Decision strategies (voting, averaging) integrate diverse outputs for reliable action selection.
- Ensemble DQN leverages deep learning ensembles to further improve performance.
- Overall, these ensemble methods enable more effective and stable RL, as demonstrated by our robot's improved maze navigation.

Thank You!

Questions are welcome.